HMM: VITERBI ALGORITHM

Iris Flower Analysis

Module 5 •Practical Report

Student Name	Hitha Pradeep Y
USN	4GW23CI017
Module	Module 5
Platform	Google Colab / Python / NumPy
Topic	Hidden Markov Model and Viterbi decoding

1. Abstract and Objectives

A Hidden Markov Model (HMM) is a probabilistic model in which an unobserved state sequence generates observable outputs. The Viterbi algorithm uses dynamic programming to identify the single most likely hidden-state sequence for a given observation sequence. This report demonstrates Viterbi decoding with measurements from the Iris flower dataset. Petal length is converted from a continuous value in centimetres into one of three categorical symbols, then decoded with a three-state categorical HMM.

Objectives

1. Load and inspect the Iris dataset.
2. Explain the observation-symbol conversion and HMM parameters.
3. Implement the Viterbi recurrence in NumPy using log probabilities.
4. Decode and compare two observation sequences.
5. Discuss assumptions, limitations, and source attribution.

Expected outcome

The program prints the observation symbols, initial/transition/emission parameters, decoded states, best log scores, and a comparison of state positions. It also produces a trellis visualization. The decoded state labels are generic model states, not Iris species predictions.

Tools used

Google Colab (execution environment), Python, NumPy (Viterbi implementation), scikit-learn (bundled Iris dataset), and Matplotlib (plots). No paid service or cloud API is required.

2. Dataset Description and Preparation

The Iris dataset contains 150 flower samples, four numeric measurements, and three known species: setosa, versicolor, and virginica. Each measurement is expressed in centimetres (cm). The dataset is commonly used for introductory classification experiments, but it is not a naturally ordered time series.

Features

Feature	Meaning	Unit
Sepal length	Length of sepal	cm
Sepal width	Width of sepal	cm
Petal length	Length of petal	cm
Petal width	Width of petal	cm

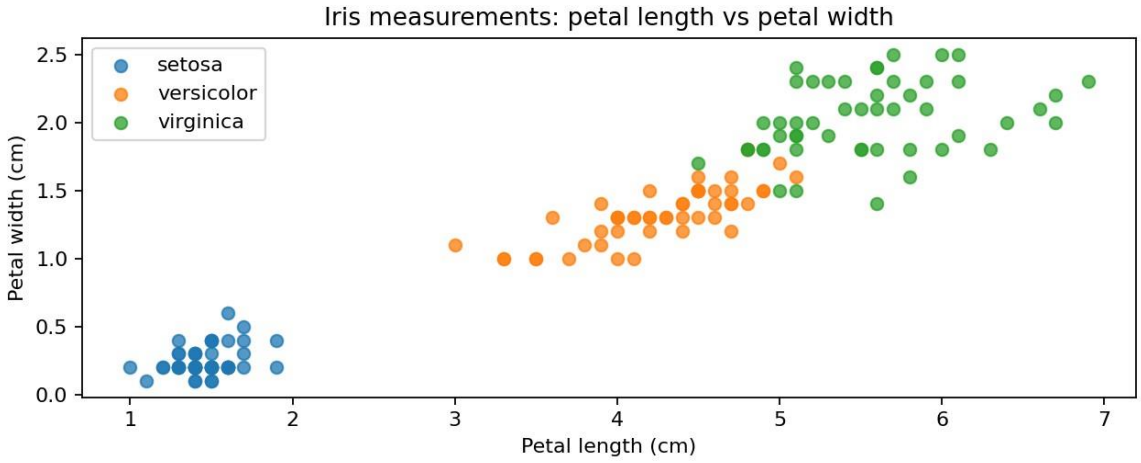


Figure 1. Petal measurements by species for dataset exploration; the supervised species labels are not used as hidden-state labels

Preparation: select petal length (feature index 2) and map it to three symbols using fixed boundaries: symbol 0 for values ≤2.0 cm, symbol 1 for values >2.0 and ≤4.5 cm, and symbol 2 for values >4.5 cm. This makes the continuous feature compatible with a categorical emission matrix.

3. Dataset Loading and Observation Encoding

The following evidence panels show the main notebook steps and their outputs. They are generated from the included reproducible code; execute the attached notebook in Google Colab to reproduce the values.

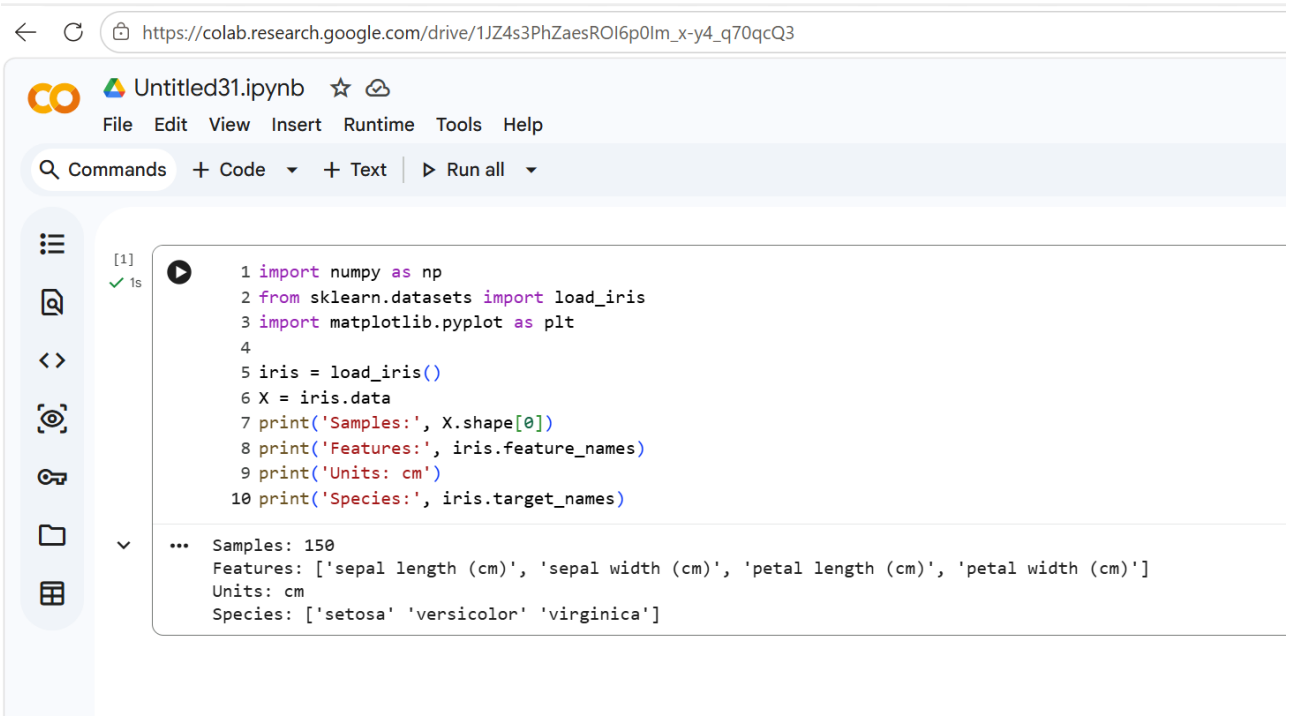


Figure 2. Notebook-style evidence: dataset loading, sample count, features and measurement units.



Figure 3. Notebook-style evidence: converting petal length into categorical observation symbols.

.

4. Hidden Markov Model and Parameters

The model contains three generic hidden states (State 0, State 1 and State 2) and three observation symbols (0, 1 and 2). The hidden-state names are arbitrary. The model parameters below are manually selected for an interpretable demonstration; they are not trained from data.

Initial-state probabilities π

```
[0.6 0.3 0.1]
```

Transition probability matrix A — A[i,j] is the probability of moving from state i to state j.

```
[[0.7  0.25 0.05]
 [0.2  0.6  0.2 ]
 [0.1  0.3  0.6 ]]
```

Emission probability matrix B — B[j,k] is the probability of observing symbol k while in state j.

```
[[0.75 0.2  0.05]
 [0.2  0.6  0.2 ]
 [0.05 0.25 0.7 ]]
```

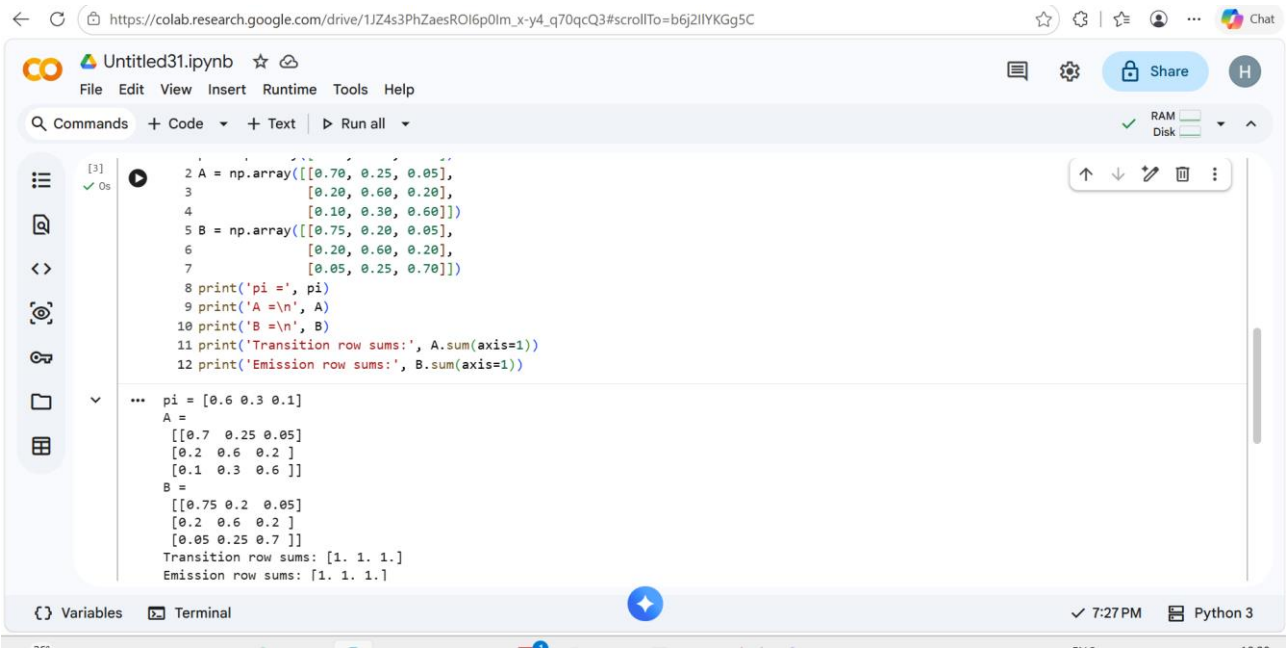


Figure 4. HMM parameters displayed with notebook output.

Each probability is non-negative and each probability row sums to 1. π sums to 1. These constraints are required for valid categorical probabilities.

5. Viterbi Algorithm

Viterbi finds the most likely single path of hidden states. It does not enumerate every possible path independently; instead, it keeps the best score ending in each state at each time step.

Log-space recurrence

Initialization: δ■(j) = log π■ + log B■(o■).

Recurrence: δ■(j) = log B■(o■) + max■ [δ■■■(i) + log A■■].

Backpointer: ψ■(j) = argmax■ [δ■■■(i) + log A■■].
Termination: choose the state with the largest final score, then follow backpointers backwards.

Logarithms turn products into sums and help prevent floating-point underflow for longer sequences. A backpointer table records which previous state gave the best score for each current state.

Implementation outline

1. Convert π, A and B to log probabilities.
2. Fill the trellis for the first observation.
3. For every later observation and every state, compute candidate predecessor scores.
4. Save the maximum score and its predecessor.
5. Select the best final state and backtrack to obtain the state sequence.

Complexity

For T observations and N hidden states, the standard Viterbi algorithm takes $O(TN^2)$ time and $O(TN)$ memory when the full trellis and backpointers are stored.

6. Trellis and Step-by-Step Interpretation

The trellis stores a log score for each possible hidden state at each observation position. At every step, the algorithm considers transitions from all previous states, adds the current emission log probability, and keeps the best predecessor.

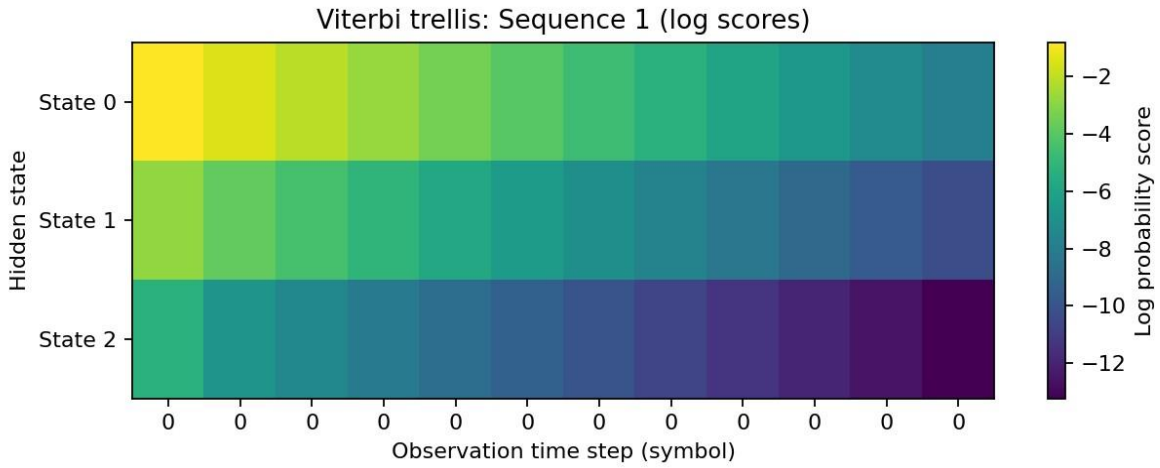


Figure 5. Trellis heatmap for Sequence 1. Columns correspond to observation positions; rows correspond to the three hidden states. Colours indicate relative log scores.

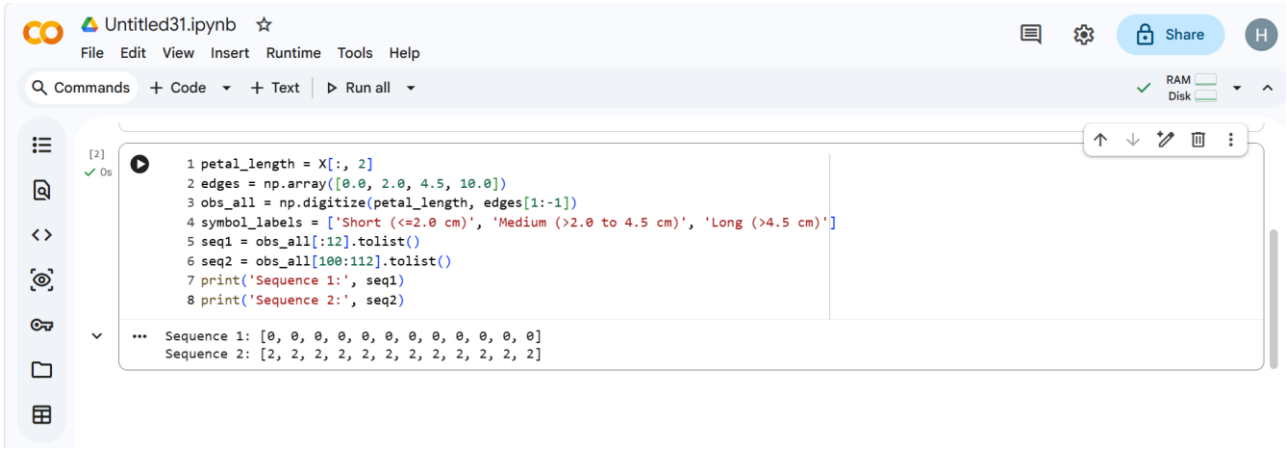
How to read it

At the first time step, the score combines the initial-state probability and the probability of the first symbol. At later steps, the highest-scoring predecessor is chosen for each candidate state. The final state is selected from the last column, and backtracking recovers the best path. A trellis score is a log probability score, so less-negative values are higher scores.

Important distinction

The best state at each time step considered independently may not produce the globally best path. Viterbi preserves the best complete prefix ending in each state, which is why backtracking is needed.

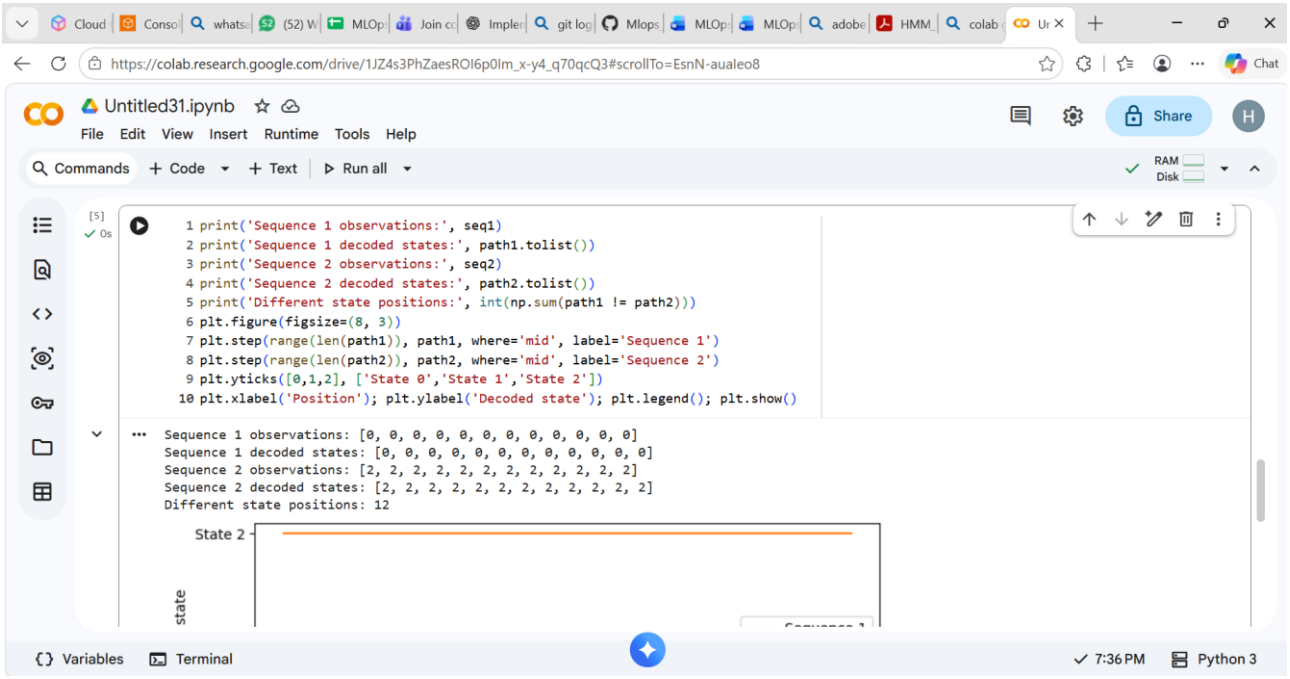
7. Execution Results and Sequence Comparison



Decoded outputs

Sequence 1 observations: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Sequence 1 decoded states: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Sequence 2 observations: [2, 2, 2, 2, 2, 2,2,2,2,2,2]
Sequence 2 decoded states: [2, 2, 2, 2, 2, 2, 2, 2,2]



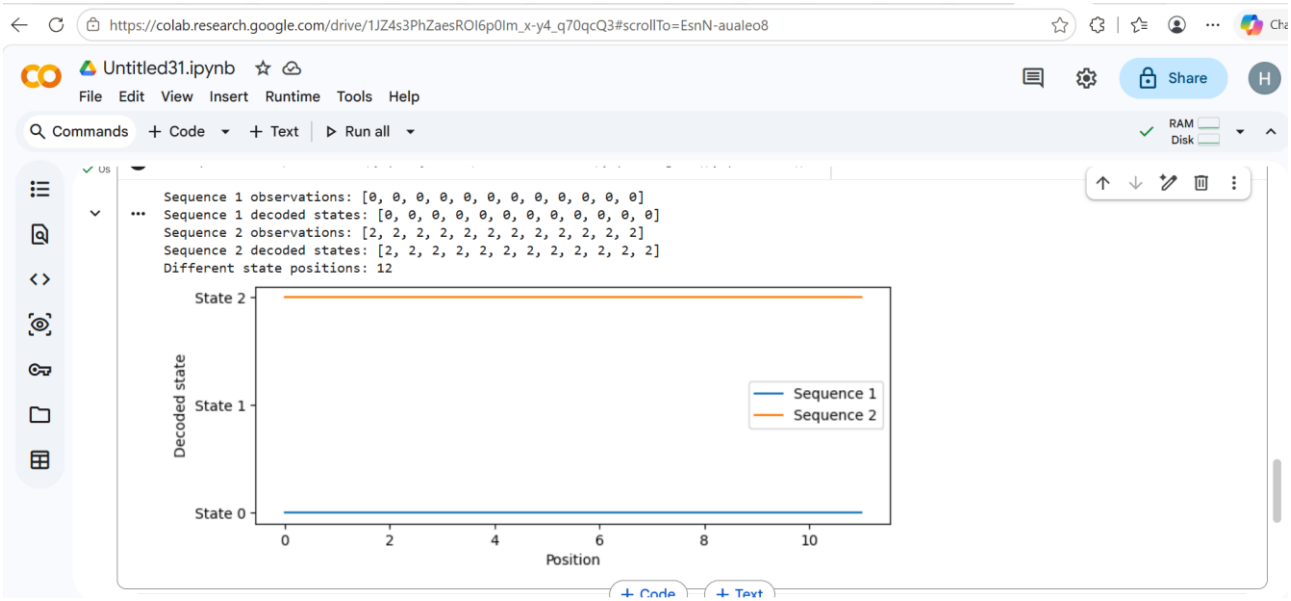


Figure 7. Comparison of decoded states by position for the two sequences.

Different observation-symbol patterns can lead to different most-likely state paths under the same HMM. Because state meanings and parameters are illustrative, this result is not a botanical conclusion and should not be evaluated as species-classification accuracy.

8. Limitations, Conclusion and Viva Notes

Limitations

- Iris is a small, classic dataset and not a naturally sequential dataset. The sequence order here is selected from roworder for demonstration.
- Petal length is discretized into three bins, which discards continuous information and makes results sensitive to binboundaries.
- Initial, transition, and emission probabilities are manually specified rather than estimated from a training sequence.
- Hidden states are generic and are not automatically equivalent to the three Iris species.
- The demonstration does not establish predictive accuracy or biological validity.

Conclusion

The experiment demonstrates how a categorical HMM represents initial-state, transition and emission probabilities, and how Viterbi uses dynamic programming and backtracking to infer the most likely hidden-state sequence. Two observation sequences were decoded and compared. The main learning outcome is the algorithmic procedure, rather than species classification.

9. Source Attribution and Reproducibility

Dataset and reference

Fisher, R. A. (1936). “The use of multiple measurements in taxonomic problems.” *Annals of Eugenics*, 7(2), 179–188.

scikit-learn developers. *load_iris — scikit-learn documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html

The dataset is accessed through the scikit-learn educational dataset loader. Consult the linked documentation and dataset metadata for exact licensing and redistribution terms when redistributing data separately. This report attributes the original Iris dataset and the software interface used to load it.

Reproducibility

Run the included `HMM_Viterbi_Iris_Analysis.ipynb` in Google Colab. The notebook defines all parameters, uses fixed bins, implements Viterbi from scratch with NumPy, and prints both decoded sequences. The included code and fixed parameters are sufficient to reproduce the outputs.

Viterbi core code

